

1)

$$E = (4 \ 2 \ 3 \ 1) \quad A = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 2 & 2 & 2 & 0 \end{pmatrix}$$

~~~~~ 22 ~~~~~

$$C = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{pmatrix} \quad R = \begin{pmatrix} 2 & 0 & 0 & 1 & \text{X} \\ 0 & 0 & 1 & 1 & \text{X} \\ 2 & 1 & 0 & 0 & 1^\circ \checkmark \end{pmatrix}$$

~~~~~ 22 ~~~~~

R: O sistema se encontra em deadlock.

2)

1. Recuperação mediante preempção:

- **Descrição:** Consiste em retirar temporariamente um recurso de seu proprietário atual e dá-lo a outro processo.
- **Viabilidade:** É difícil ou impossível na prática para muitos recursos. Depende muito da natureza do recurso (fácil para CPU ou memória, impossível para uma impressora no meio de uma impressão sem estragar o trabalho). Frequentemente requer intervenção manual.

2. Recuperação mediante retrocesso:

- **Descrição:** O sistema periodicamente cria *checkpoints* dos processos (salvando estado de memória e recursos). Quando um deadlock é detectado, um processo é revertido a um estado anterior (antes de adquirir o recurso bloqueante) e reiniciado.
- **Viabilidade:** Viável, mas custoso. Requer que o sistema tenha mecanismos de checkpoint implementados, o que consome processamento e espaço em disco. O trabalho realizado entre o checkpoint e o deadlock é perdido.

3. Recuperação mediante a eliminação de processos:

- **Descrição:** A forma mais bruta. Envolve matar um ou mais processos envolvidos no ciclo de deadlock (ou um processo externo que possua os recursos necessários).
- **Viabilidade:** É a mais simples de implementar, mas pode causar inconsistência de dados (por exemplo, se o processo estava atualizando um banco de dados). É preferível matar processos que podem ser reexecutados sem danos, como uma compilação, em vez de processos transacionais.

3)

Não, não existe nenhuma circunstância que gere deadlock neste caso. Imagine o pior cenário possível, onde todos os processos retêm recursos mas ainda não conseguem terminar. Como cada um dos 3 processos precisa de no máximo 2 instâncias, o travamento só aconteceria se todos tivessem 1 instância e estivessem esperando pela segunda. Se os três processos pegarem 1 recurso cada, teremos 3 recursos ocupados. Como o sistema possui um total de 4 recursos, obrigatoriamente sobrarão uma instância livre. Esse recurso restante pode ser alocado para qualquer um dos processos, permitindo que ele complete sua execução e libere seus recursos para os demais, o que torna o deadlock matematicamente impossível.

4)

O Algoritmo do Banqueiro modela o sistema operacional como um banqueiro que lida com um grupo de clientes, representados pelos processos, e uma quantidade limitada de dinheiro, que simboliza os recursos disponíveis. Nesse modelo, cada processo deve declarar antecipadamente o número máximo de recursos de que precisará durante sua execução.

Quando um processo solicita um recurso, o sistema verifica se conceder essa solicitação levará o sistema a um estado seguro. Um estado é considerado seguro quando existe uma sequência de execução na qual todos os processos podem terminar com sucesso, mesmo que todos solicitem seus recursos máximos simultaneamente. Se o estado resultante da concessão for seguro, a solicitação é atendida. Por outro lado, se for inseguro, ou seja, se não houver garantia de que haverá recursos suficientes para que pelo menos um processo termine e libere seus recursos futuros, a solicitação é negada ou o processo é colocado em espera até que a concessão possa ser feita com segurança.

Essencialmente, o algoritmo evita o deadlock garantindo que o sistema nunca entre em um estado onde não possa satisfazer as necessidades futuras de pelo menos um processo, permitindo assim que este libere seus recursos e possibilite a continuação dos demais processos.